ICT Academy of Kerala · Python Programming

Student Result Management System

Python Functions · 1-hour case study

Submit your work on Paatshala as a GitHub repository link

How to work

Step	What to do
1	Create a project folder in VS Code and activate your virtual environment. The terminal must show the environment name before you run anything.
2	Create the file `result_system.py` and copy the starter data exactly as given.
3	Do the tasks in order. Run your program after every task and compare it with the expected output.
4	Your output must match the expected output exactly, including spaces and alignment.
5	Call your trainer when you finish, and try the challenge if time is left.

The scenario

A training centre keeps the marks of its students in a Python **dictionary**: the key is the student id and the value is another dictionary with the student's name and a dictionary of subject marks. The office staff currently work out totals, averages and grades by hand. **You will turn that work into functions** so the centre can calculate results, update marks, add and remove students, and print a report with one call.
File name: `result_system.py` **Time:** 60 minutes

What you will practise

- Dictionaries: nested dictionaries, `.values()`, `.items()`, `.update()`, `.pop()`, `.setdefault()`
- Loops, `if / elif / else` and calculating totals and averages
- Functions: parameters, `return`, default argument (`pass_mark=40`), `**kwargs`, functions calling other functions
- Sorting a list of tuples and formatted output with f-strings

STARTER DATA (INPUT IN DICTIONARY FORMAT; COPY EXACTLY)

```
students = {
    "S101": {"name": "Anu",   "marks": {"Python": 88, "Maths": 76, "Stats": 92}},
    "S102": {"name": "Binu",  "marks": {"Python": 45, "Maths": 38, "Stats": 52}},
    "S103": {"name": "Cathy", "marks": {"Python": 95, "Maths": 91, "Stats": 89}},
    "S104": {"name": "Dev",   "marks": {"Python": 60, "Maths": 72, "Stats": 55}},
    "S105": {"name": "Esha",  "marks": {"Python": 72, "Maths": 65, "Stats": 80}},
}
```

Functions to write

Function	What it does
calculate_total(marks)	returns the sum of all marks in the marks dictionary
calculate_average(marks)	returns the average rounded to 2 decimals (use `calculate_total`)
get_grade(average)	returns `"A"` for 90 and above, `"B"` for 75 and above, `"C"` for 60 and above, `"D"` for 40 and above, otherwise `"F"`
check_result(marks, pass_mark=40)	returns `"Pass"` only if **every** subject is at least the pass mark, otherwise `"Fail"`
student_summary(student)	takes one student dictionary and returns a new dictionary with name, total, average, grade and result
add_student(data, student_id, name, **marks)	adds a student with the marks given as keyword arguments and returns `True`; if the id already exists it changes nothing and returns `False`
update_marks(data, student_id, **changes)	updates the marks given as keyword arguments (for example `Maths=55`) and returns `True`; returns `False` if the id does not exist
remove_student(data, student_id)	removes the student and returns `True`; returns `False` if the id does not exist
print_report(data)	prints the report table, students sorted by average, highest first (build a list of `(average, id)` tuples and use `sort(reverse=True)`)
class_statistics(data)	returns a dictionary with the topper's name, the pass percentage (1 decimal) and a dictionary of the average mark of each subject (1 decimal)
main()	prints each task heading and calls the functions; end the file with `main()`

Every function should **return** its result (except `print_report` and `main`, which print). Do not use global variables inside the functions; pass the dictionary in as a parameter.

Tasks and expected output

Task 1 · Summary of one student

Print the summary dictionary of student S101. Then print `check_result` for the marks of S102 with the default pass mark, and again with `pass_mark=35`.

EXPECTED OUTPUT

```
Task 1: One student
{'name': 'Anu', 'total': 256, 'average': 85.33, 'grade': 'B', 'result': 'Pass'}
Fail
Pass
```

Task 2 · Report of the class

Print the report of all students. Widths: ID 6, Name 8, Total 6, Avg 8 (two decimals), Grade 7, then two spaces and the result. A line of 42 dashes under the heading. Sort by average, highest first: collect `(average, id)` tuples in a list and call `sort(reverse=True)`.

EXPECTED OUTPUT

```
Task 2: Report
ID    Name     Total     Avg  Grade  Result
------------------------------------------
S103  Cathy      275   91.67      A  Pass
S101  Anu        256   85.33      B  Pass
S105  Esha       217   72.33      C  Pass
S104  Dev        187   62.33      C  Pass
S102  Binu       135   45.00      D  Fail
```

Task 3 · Update marks

Binu has re-checked his Maths paper and the mark is corrected to 55. Update it with `update_marks` and print the result. Try to update the unknown id S999 and print the result. Print the new summary of S102.

EXPECTED OUTPUT

```
Task 3: Update marks
True
False
{'name': 'Binu', 'total': 152, 'average': 50.67, 'grade': 'D', 'result': 'Pass'}
```

Task 4 · Add a student

Add a new student S106, Farhan, with Python 70, Maths 35 and Stats 74 (use keyword arguments) and print the result. Try to add S101 again with the name Zara and print the result. Print the new student's dictionary.

EXPECTED OUTPUT

```
Task 4: Add student
True
False
{'name': 'Farhan', 'marks': {'Python': 70, 'Maths': 35, 'Stats': 74}}
```

Task 5 · Remove a student

Remove S104 (Dev) and print the result. Try to remove the same id again and print the result. Print how many students are left.

EXPECTED OUTPUT

```
Task 5: Remove student
True
False
5
```

Task 6 · Final report and class statistics

Print the report again, then use `class_statistics` to print the topper, the pass percentage and the subject averages.

EXPECTED OUTPUT

```
Task 6: Final report and statistics
ID    Name     Total     Avg  Grade  Result
----------------------------------------
S103  Cathy      275   91.67      A  Pass
S101  Anu        256   85.33      B  Pass
S105  Esha       217   72.33      C  Pass
S106  Farhan     179   59.67      D  Fail
S102  Binu       152   50.67      D  Pass
Topper: Cathy
Pass percentage: 80.0
Subject averages: {'Python': 74.0, 'Maths': 64.4, 'Stats': 77.4}
```

Print the heading lines (for example `Task 1: One student`) exactly as shown, with a blank line between tasks. The tasks run in order, so each task changes the data used by the next one. Write and run one function at a time.

Suggested time

Tasks	Minutes
Set up folder, virtual environment, copy the data	5
Write the first five functions (calculation and summary) and Tasks 1 and 2	20
Functions that change the data (add, update, remove) and Tasks 3 to 5	20
class_statistics, final report (Task 6)	10
Check output, push to GitHub, submit link on Paatshala	5

Challenge (optional)

- Add `subject_topper(data, subject)` that returns the name of the student with the highest mark in a subject.
- Add `students_below(data, limit=50)` that returns the names of students whose average is below the limit.
- Add a grade distribution: a dictionary that counts how many students got each grade.

Submission

- Create a GitHub repository and upload `result_system.py`.

- Submit the repository link on Paatshala.